Proyecto Final de Análisis Numérico

Interpolación de la población de Ucrania (1994–2023)

Paula Villamil González, Carolina Ujueta Ricardo
Pontificia Universidad Javeriana
Departamento de Matemáticas
Análisis Numérico

noviembre de 2026

1. Introducción

El objetivo del proyecto es comparar el desempeño de los polinomios de Lagrange y de los trazadores cúbicos para interpolar un conjunto numeroso de datos. Para ello se aproxima la población de Ucrania, el país asignado, entre 1994 y 2023 a partir de datos discretos.

El conjunto de datos tiene 30 registros anuales. De ellos, 25 se usan como nodos de interpolación (las filas sin color en el Excel) y los otros 5, que están resaltados (subrayados en el enunciado), se reservan como datos de prueba. Estos últimos se aproximan con cada método y luego se comparan con el valor real. Los años de prueba son 1994, 2002, 2009, 2017 y 2023. Dos de ellos, 1994 y 2023, quedan fuera del rango de los nodos (1995–2022), de modo que en ellos se extrapola. Los otros tres, 2002, 2009 y 2017, quedan entre nodos y se interpola.

Se comparan dos métodos:

- el polinomio interpolador de Lagrange, de grado 24, que usa un único polinomio para todo el intervalo, y
- el trazador cúbico natural, que usa un polinomio cúbico distinto en cada tramo y exige que la curva sea suave en los nodos.

La calidad de cada método se mide con el error relativo

$$E_r = \frac{|y_{\text{exacto}} - y_{\text{aprox}}|}{|y_{\text{exacto}}|},$$

que permite comparar errores de distintos años sin importar la magnitud de la población.

2. Implementación de los algoritmos

Se eligió Python 3 por tres razones. Su sintaxis es clara y se parece a la notación matemática de los algoritmos. Sus números de punto flotante son de doble precisión (64 bits), suficientes para trabajar con poblaciones de ocho cifras. Además, tiene librerías para leer el Excel (`openpyxl`) y para graficar (`matplotlib`). Las dependencias están en `requirements.txt` y se instalan con `pip install -r requirements.txt`. Los algoritmos de interpolación no usan ninguna librería numérica: se programaron desde cero con listas y ciclos de Python, para que cada paso se pueda seguir y explicar.

El proyecto es modular. Cada algoritmo vive en su propio archivo y se puede ejecutar por separado con `python Lagrange/lagrange.py` o `python Trazadores/trazadores.py`. Ambos importan la lectura de datos desde `datos.py`, así que el Excel se lee en un solo lugar y los dos métodos usan exactamente los mismos 25 nodos.

2.1. Estructura del programa

- `datos.py`: lee el archivo Excel y separa nodos y datos de prueba.
- `lagrange.py`: polinomio interpolador de Lagrange.
- `trazadores.py`: trazador cúbico natural (algoritmo de Thomas).
- `graficas.py`: genera las curvas y los diagramas de error en formato PNG.

2.2. Captura y organización de datos (`datos.py`)

Para el desarrollo del Proyecto se descargó exclusivamente la hoja correspondiente a Ucrania, ya que fue el país asignado para el desarrollo del taller. El Excel tiene el año en la columna B y la población en la columna C. La función `cargar_datos` recorre las filas desde la segunda, porque la primera es el encabezado, y descarta las filas vacías. Para decidir si una fila es nodo o dato de prueba revisa el relleno de la celda del año. Si es sólido, la fila está resaltada y es un dato de prueba. Si no tiene relleno, es un nodo. La función devuelve cuatro listas: los años y las poblaciones de los nodos (`x`, `y`), y los años y las poblaciones de los datos de prueba (`xr`, `yr`). De esta forma, ningún dato de prueba entra a la construcción de los polinomios.

```
def cargar_datos(ruta=EXCEL):
    ws = openpyxl.load_workbook(ruta).active
    x, y, xr, yr = [], [], [], []
    for fila in range(2, ws.max_row + 1):
        anio, pob = ws.cell(fila, 2), ws.cell(fila, 3)
        if anio.value is None or pob.value is None:
            continue
        # Celdas con color -> datos resaltados a aproximar
        if anio.fill.fill_type == "solid":
            xr.append(anio.value)
            yr.append(pob.value)
        else:  # Celdas sin color -> nodos para interpolar
            x.append(anio.value)
            y.append(pob.value)
    return x, y, xr, yr
```

2.3. Algoritmo 1: Polinomio de Lagrange (`lagrange.py`)

Dados $n = 25$ nodos (x_k, y_k), el polinomio interpolador de grado $n - 1 = 24$ es

$$L_k(x) = \prod_{j \neq k} \frac{x - x_j}{x_k - x_j}, \qquad P(x) = \sum_{k=0}^{n-1} y_k \, L_k(x).$$

Cada base L_k vale 1 en el nodo x_k y 0 en todos los demás nodos. Por eso $P(x_k) = y_k$: el polinomio pasa exactamente por los 25 puntos. La función `base_lagrange` calcula una base $L_k(x)$ con un ciclo que multiplica los factores $(x - x_j)/(x_k - x_j)$ para todo $j \neq k$. La función `lagrange` suma esas bases ponderadas por las poblaciones y_k. Cada evaluación cuesta $O(n^2)$ operaciones, porque hay n bases y

cada una necesita $n-1$ productos. Para el proyecto esto es despreciable. La función `error_relativo` implementa la fórmula de la introducción.

```python
def base_lagrange(x, nodos, k):
    """L_k(x) = prod_{j != k} (x - x_j) / (x_k - x_j)"""
    L = 1.0
    for j, xj in enumerate(nodos):
        if j != k:
            L *= (x - xj) / (nodos[k] - xj)
    return L

def lagrange(x, nodos, valores):
    """P(x) = sum_k y_k * L_k(x), con todos los nodos dados."""
    return sum(valores[k] * base_lagrange(x, nodos, k)
               for k in range(len(nodos)))

def error_relativo(exacto, aprox):
    return abs(exacto - aprox) / abs(exacto)
```

2.4. Algoritmo 2: Trazador cúbico natural (`trazadores.py`)

Los polinomios de interpolación de grado alto tienden a presentar oscilaciones severas cerca de los extremos del intervalo (fenómeno de Runge). Para evitar esta inestabilidad, se implementó el trazador cúbico natural basado en el Algoritmo 3.4 del libro de Burden y Faires [1]. En cada tramo o subintervalo $[x_j, x_{j+1}]$ para $j = 0, \ldots, n-1$, se define un polinomio cúbico independiente de la forma:

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3.$$

Con $n+1 = 25$ nodos de interpolación existen $n = 24$ tramos y, por lo tanto, $4 \cdot 24 = 96$ coeficientes desconocidos por determinar. Las condiciones matemáticas que garantizan la suavidad y continuidad del trazador son:

- **Interpolación en nodos:** $S_j(x_j) = y_j$ y $S_j(x_{j+1}) = y_{j+1}$, asegurando que la curva pase exactamente por todos los datos conocidos.
- **Continuidad de derivadas (S' y S''):** La primera y la segunda derivada coinciden a ambos lados de cada nodo interior x_j ($j = 1, \ldots, n-1$), de modo que la función interpolante carece de picos o cambios bruscos de curvatura.
- **Condiciones de frontera naturales:** $S''(x_0) = S''(x_n) = 0$, que imponen curvatura nula en los extremos y completan las ecuaciones requeridas.

Directamente se establece $a_j = y_j$. Al sustituir las condiciones de continuidad de derivadas y fronteras naturales, el problema se reduce a resolver un sistema lineal de ecuaciones tridiagonal de orden $(n+1) \times (n+1)$ para los coeficientes cuadráticos c_j, de la forma $A\mathbf{c} = \boldsymbol{\alpha}$, donde el vector del lado derecho está dado por:

$$\alpha_i = \tfrac{3}{h_i}(a_{i+1} - a_i) - \tfrac{3}{h_{i-1}}(a_i - a_{i-1}), \qquad h_i = x_{i+1} - x_i \quad (i = 1, \ldots, n-1).$$

Para resolver el sistema tridiagonal de manera óptima, no se recurre a la eliminación gaussiana estándar ni a la inversión matricial tradicional, las cuales requieren una complejidad computacional de $\mathcal{O}(n^3)$ operaciones numéricas y una memoria de $\mathcal{O}(n^2)$. En su lugar, se emplea el **Algoritmo de Thomas** (también conocido como *Tri-Diagonal Matrix Algorithm* o TDMA), el cual constituye

una versión especializada y eficiente de la **Factorización de Crout** para matrices tridiagonales diagonales dominantes [1, 3].

La Factorización de Crout descompone la matriz tridiagonal A en el producto de dos matrices triangulares $A = LU$, donde L es una matriz triangular inferior y U es una matriz triangular superior con unos en su diagonal principal. El Algoritmo de Thomas aprovecha la estructura de ceros estructurales de A para efectuar esta descomposición y resolver el sistema mediante dos etapas de complejidad lineal $\mathcal{O}(n)$ en tiempo y memoria:

1. **Barrido hacia adelante (Factorización LU y solución de $L\mathbf{z} = \boldsymbol{\alpha}$):** Se imponen las condiciones iniciales de frontera natural en $i = 0$ ($l_0 = 1$, $\mu_0 = 0$, $z_0 = 0$). Para $i = 1, \ldots, n-1$, se calculan los elementos diagonales de L (l_i), los elementos de la superdiagonal de U (μ_i) y las incógnitas intermedias del vector $\mathbf{z}$ (z_i):

$$l_i = 2(x_{i+1} - x_{i-1}) - h_{i-1}\mu_{i-1}, \qquad \mu_i = \frac{h_i}{l_i}, \qquad z_i = \frac{\alpha_i - h_{i-1}z_{i-1}}{l_i}.$$

   En el nodo final $i = n$, la condición natural establece $l_n = 1$ y $z_n = 0$, asegurando $c_n = 0$.

2. **Sustitución hacia atrás (Solución de $U\mathbf{c} = \mathbf{z}$ y coeficientes del trazador):** Recorriendo en orden descendente desde $j = n - 1$ hasta 0, se obtienen los coeficientes cuadráticos c_j y, consecuentemente, los coeficientes lineal b_j y cúbico d_j:

$$c_j = z_j - \mu_j c_{j+1}, \qquad b_j = \frac{a_{j+1} - a_j}{h_j} - \frac{h_j(c_{j+1} + 2c_j)}{3}, \qquad d_j = \frac{c_{j+1} - c_j}{3h_j}.$$

El código implementado en `trazadores.py` sigue rigurosamente esta secuencia matemática:

Listing 1: Algoritmo de construcción del trazador cúbico natural

```
def construir_trazador_cubico_natural(nodos, valores):
    puntos = sorted(zip(nodos, valores))
    x = [float(p[0]) for p in puntos]
    a = [float(p[1]) for p in puntos]
    n = len(x) - 1

    # Anchos de los subintervalos h_i
    h = [x[i + 1] - x[i] for i in range(n)]

    # Vector alpha del sistema tridiagonal
    alpha = [0.0] * (n + 1)
    for i in range(1, n):
        alpha[i] = (3.0 / h[i]) * (a[i + 1] - a[i]) \
                 - (3.0 / h[i - 1]) * (a[i] - a[i - 1])

    # Algoritmo de Thomas (Factorización de Crout)
    l = [1.0] + [0.0] * n
    mu = [0.0] * (n + 1)
    z = [0.0] * (n + 1)
    for i in range(1, n):
        l[i] = 2.0 * (x[i + 1] - x[i - 1]) - h[i - 1] * mu[i - 1]
        mu[i] = h[i] / l[i]
        z[i] = (alpha[i] - h[i - 1] * z[i - 1]) / l[i]

    # Fronteras naturales: c_0 = 0 y c_n = 0
    l[n], z[n] = 1.0, 0.0
    c = [0.0] * (n + 1)
    b = [0.0] * n
    d = [0.0] * n
```

```
30
31      # Sustitución hacia atrás
32      for j in range(n - 1, -1, -1):
33          c[j] = z[j] - mu[j] * c[j + 1]
34          b[j] = (a[j + 1] - a[j]) / h[j] \
35               - h[j] * (c[j + 1] + 2.0 * c[j]) / 3.0
36          d[j] = (c[j + 1] - c[j]) / (3.0 * h[j])
37
38      return x, a[:n], b, c[:n], d
```

Una vez calculados los coeficientes (a, b, c, d), la función `evaluar_trazador` localiza el subintervalo $[x_j, x_{j+1}]$ correspondiente al año solicitado y evalúa el polinomio cúbico respectivo. Si el año a evaluar se encuentra fuera del rango nodal $[x_0, x_n]$, se emplea el primer o último polinomio según corresponda (extrapolación por tramos), como sucede para los años 1994 y 2023.

Listing 2: Evaluación del trazador en un punto dado

```
1  def evaluar_trazador(x_eval, x, a, b, c, d):
2      n = len(b)
3      # Selección del tramo polinomial j
4      if x_eval <= x[0]:
5          j = 0
6      elif x_eval >= x[-1]:
7          j = n - 1
8      else:
9          j = 0
10         for i in range(n):
11             if x[i] <= x_eval <= x[i + 1]:
12                 j = i
13                 break
14     dx = x_eval - x[j]
15     return a[j] + b[j]*dx + c[j]*dx**2 + d[j]*dx**3
```

Cada módulo de interpolación cuenta con una función `main` que carga los datos mediante `datos.py`, construye el modelo (Lagrange o trazador), lo evalúa en los cinco años resaltados de prueba e imprime una tabla comparativa con los valores exactos, aproximados y sus correspondientes errores relativos. Finalmente, `graficas.py` importa estas mismas funciones para trazar las curvas continuas y generar los gráficos de error, garantizando absoluta consistencia entre los resultados tabulares y los gráficos presentados.

2.5. Tabla de variables principales

Variable	Tipo	Descripción
`nodos` / `x`	`list[float]`	Años conocidos x_i usados como nodos ($n = 25$).
`valores` / `a`	`list[float]`	Población conocida $y_i = a_i$ en los nodos.
`resaltados`	`list[int]`	Años de prueba (5 datos) para validar los métodos.
`h`	`list[float]`	Distancias entre nodos consecutivos $h_i = x_{i+1} - x_i$.
`alpha`	`list[float]`	Lado derecho del sistema tridiagonal.
`l, mu, z`	`list[float]`	Vectores auxiliares de la factorización de Crout.
`b, c, d`	`list[float]`	Coeficientes lineal, cuadrático y cúbico de cada tramo.

Cuadro 1: Variables principales del sistema de interpolación.

2.6. Diagramas de flujo

Los flujos lógicos de los módulos se modelaron en Mermaid y se exportaron a PDF para mantener la nitidez en la impresión. El primero muestra cómo se leen y separan los datos. Los otros dos describen el cálculo de cada método.

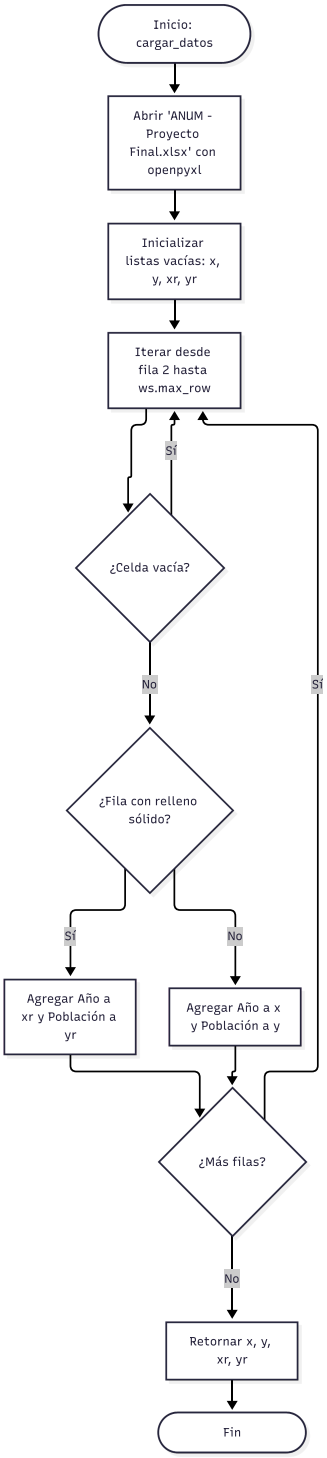


Figura 1: Flujo de datos de `cargar_datos`.

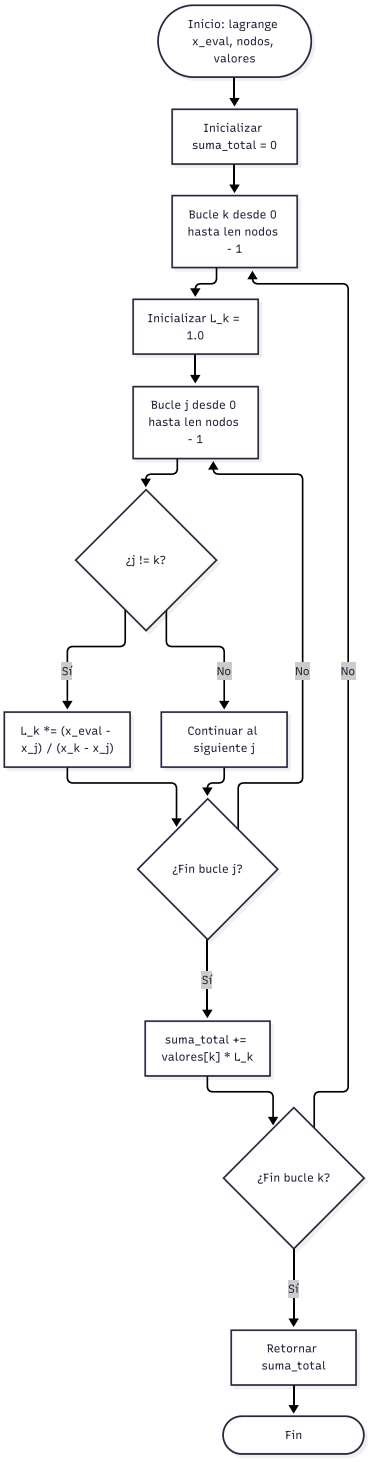


Figura 2: Flujo del algoritmo de Lagrange.

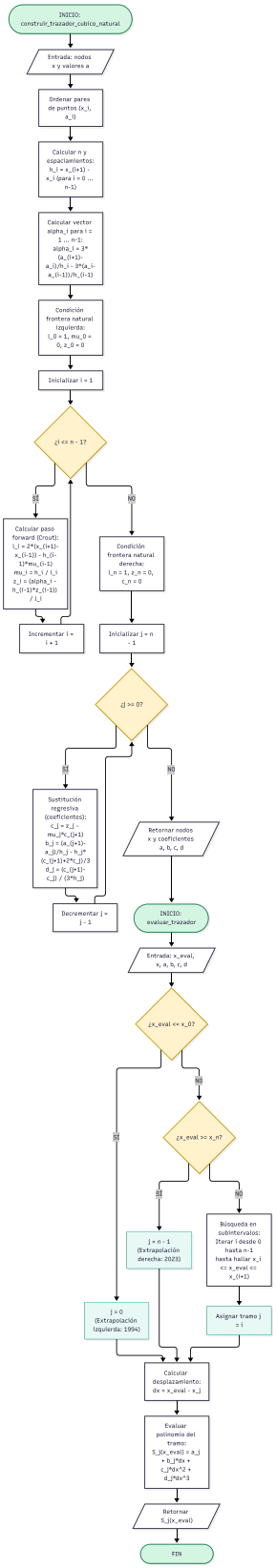


Figura 3: Flujo del trazador cúbico natural.

2.7. Instrucciones para ejecutar el código

El código necesita Python 3.9 o superior y las librerías `openpyxl` y `matplotlib`, que se instalan con

```
pip install -r requirements.txt
```

Todos los comandos se ejecutan desde la carpeta principal del proyecto, donde están `datos.py` y el archivo Excel:

- `python lagrange.py` construye el polinomio de Lagrange con los 25 nodos e imprime, para los 5 años resaltados, el valor exacto, el valor aproximado y el error relativo.
- `python trazadores.py` hace lo mismo con el trazador cúbico natural.
- `python graficas.py` genera todas las gráficas de este informe en la carpeta `graficas/`, que se crea sola.

Los archivos `lagrange.py` y `trazadores.py` solo necesitan `datos.py` y el Excel, por lo que se pueden correr sin generar las gráficas.

3. Resultados

3.1. Resultados por método

Se evaluaron ambos métodos en los 5 años de prueba, usando solo los 25 nodos. Las tablas siguientes muestran, para cada año, el valor exacto, la aproximación y el error relativo. Los valores son los que imprimen los programas `lagrange.py` y `trazadores.py`.

Año	Valor exacto	Aproximación (Lagrange)	Error relativo
1994	52 076 000	−21 736 816 040,49	418,4056386913
2002	48 677 280	48 695 338,85	0,0003709914
2009	46 623 731	46 608 582,57	0,0003249082
2017	45 436 037	45 753 319,55	0,0069830595
2023	37 732 833	−33 412 276 260,07	886,4960946100

Cuadro 2: Interpolación de Lagrange (grado 24).

Año	Valor exacto	Aproximación (trazador)	Error relativo
1994	52 076 000	52 104 959,00	0,0005560911
2002	48 677 280	48 683 629,82	0,0001304474
2009	46 623 731	46 618 926,17	0,0001030555
2017	45 436 037	45 420 397,18	0,0003442162
2023	37 732 833	37 798 894,00	0,0017507564

Cuadro 3: Trazador cúbico natural (24 tramos).

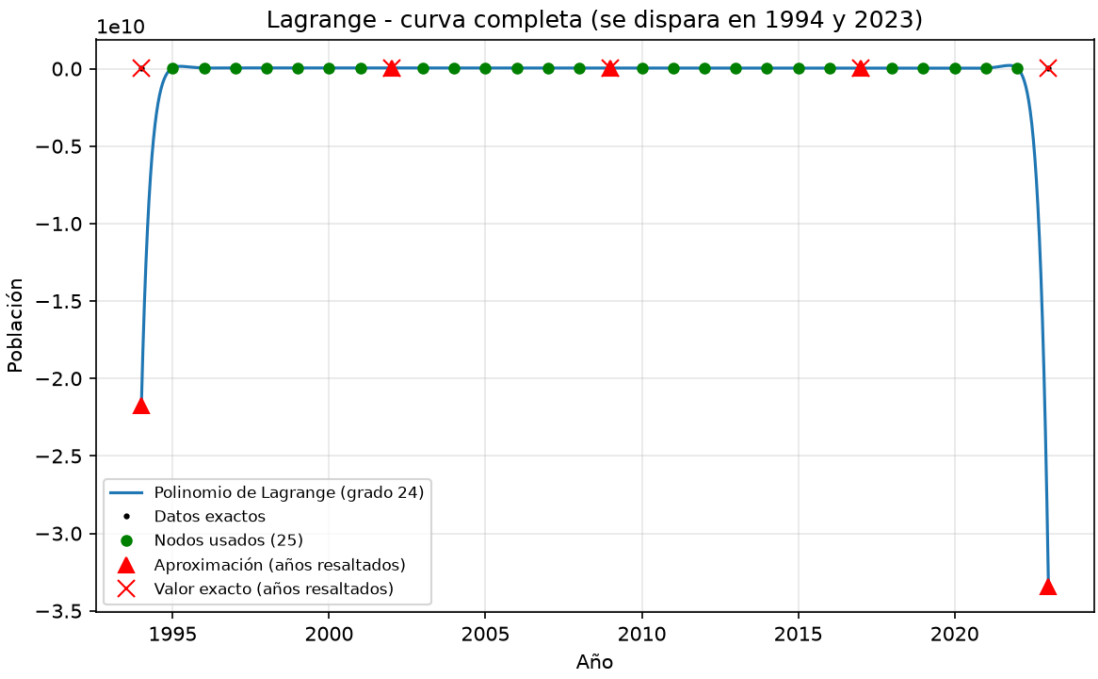


Figura 4: Polinomio de Lagrange en todo el rango 1994–2023: diverge en los extremos.

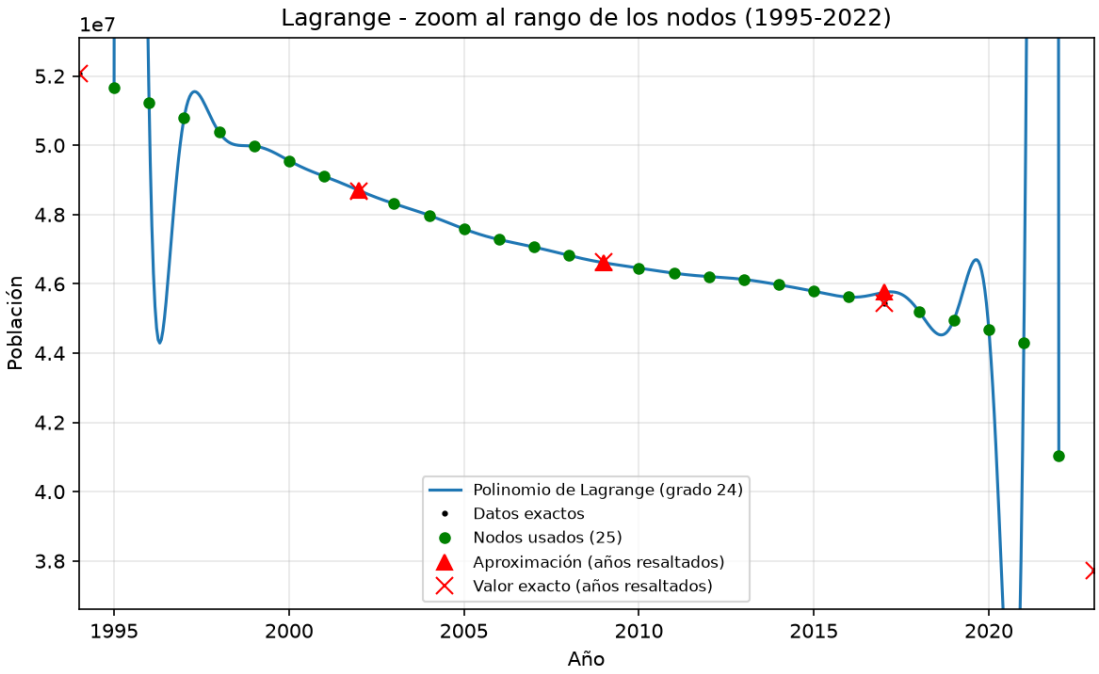


Figura 5: Lagrange ampliado al rango de los nodos (1995–2022).

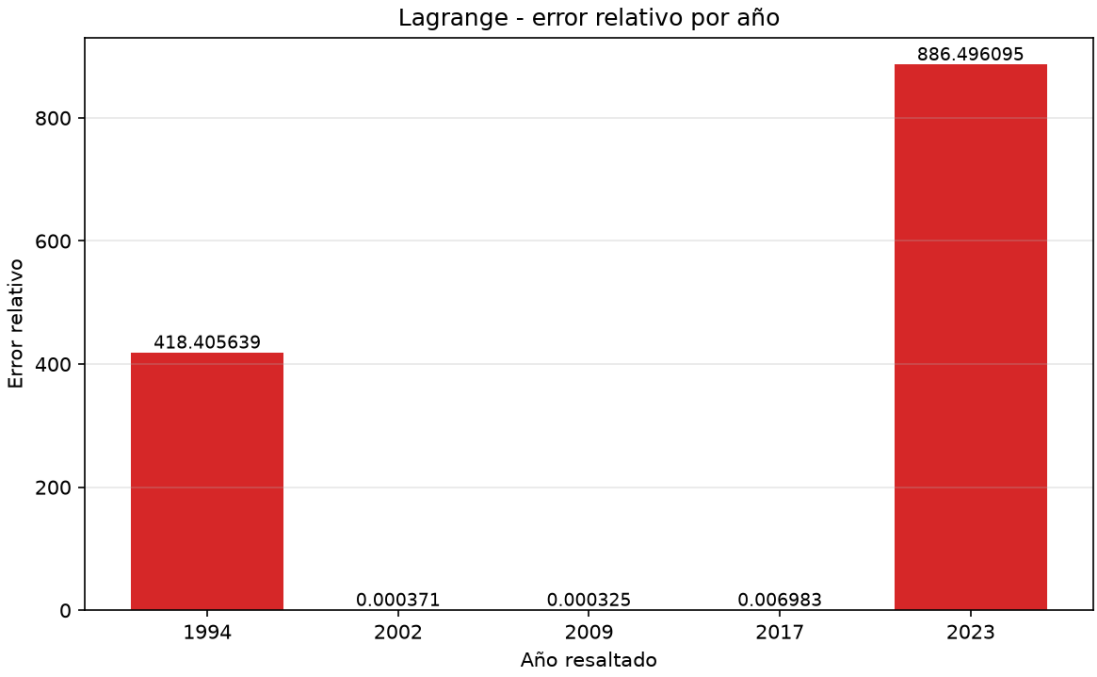


Figura 6: Error relativo de Lagrange por año de prueba.

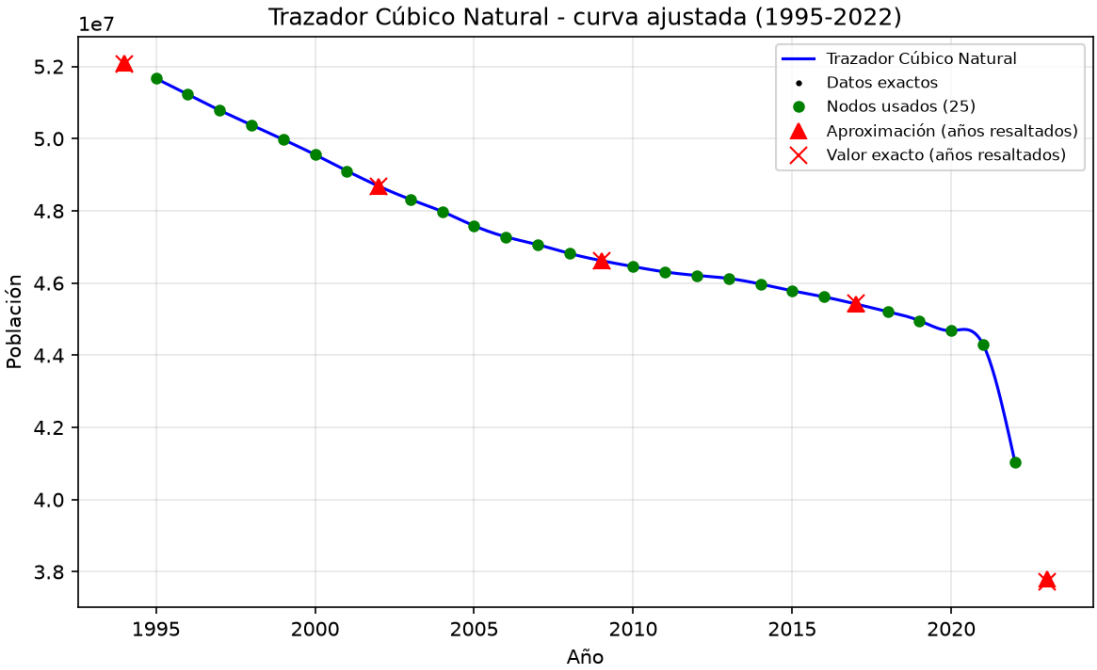


Figura 7: Trazador cúbico natural ajustado a los nodos.

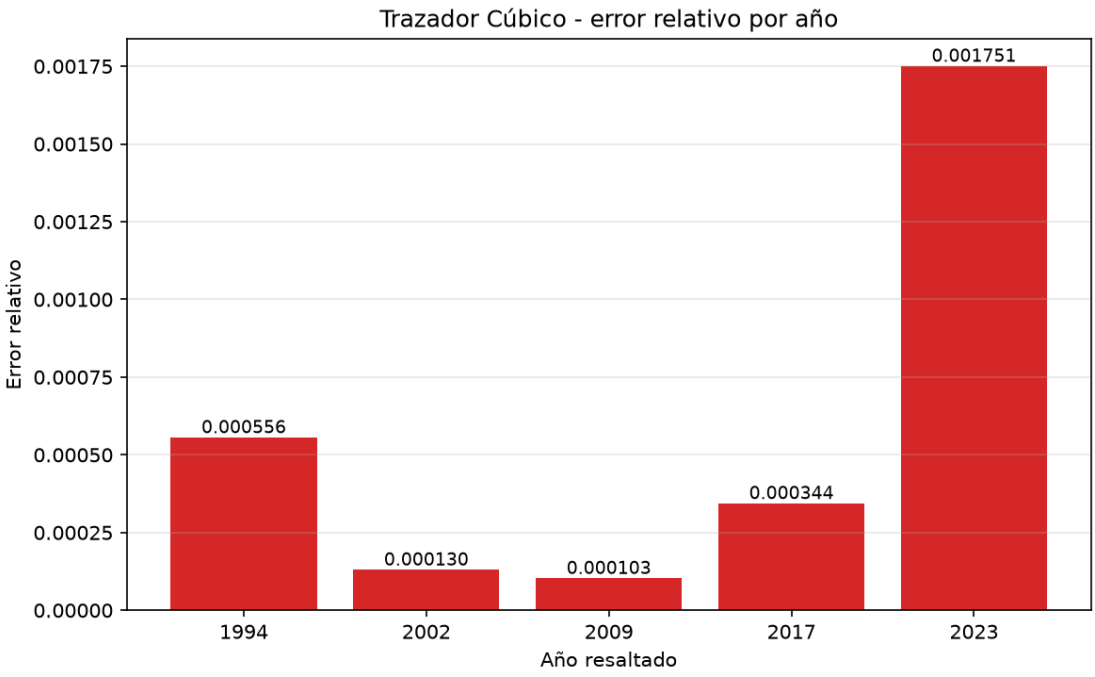


Figura 8: Error relativo del trazador cúbico por año de prueba.

3.2. Comparación entre ambos métodos

Año	Valor exacto	Error rel. Lagrange	Error rel. trazador
1994	52 076 000	418,4056	0,000556
2002	48 677 280	0,000371	0,000130
2009	46 623 731	0,000325	0,000103
2017	45 436 037	0,006983	0,000344
2023	37 732 833	886,4961	0,001751

Cuadro 4: Comparación del error relativo de ambos métodos en los años resaltados.

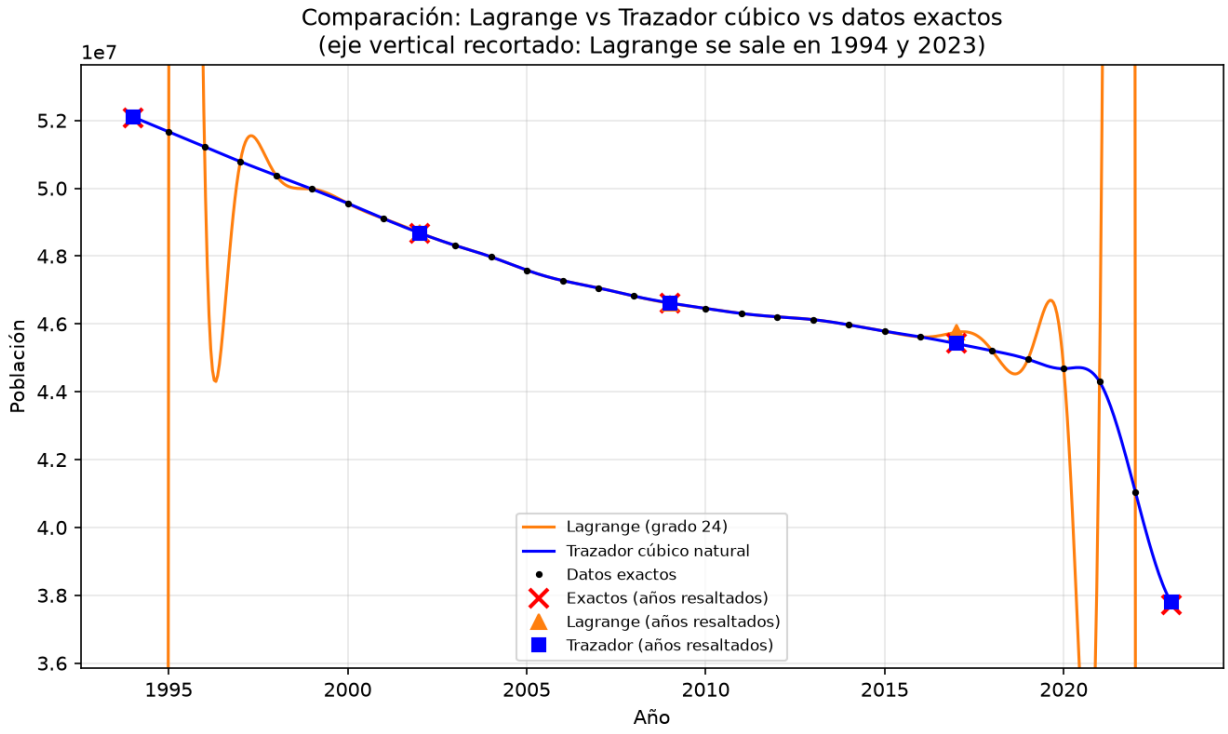


Figura 9: Lagrange, trazador cúbico y valores exactos en una sola gráfica. El eje vertical está recortado para poder ver los datos; Lagrange queda fuera del rango mostrado en 1994 y 2023.

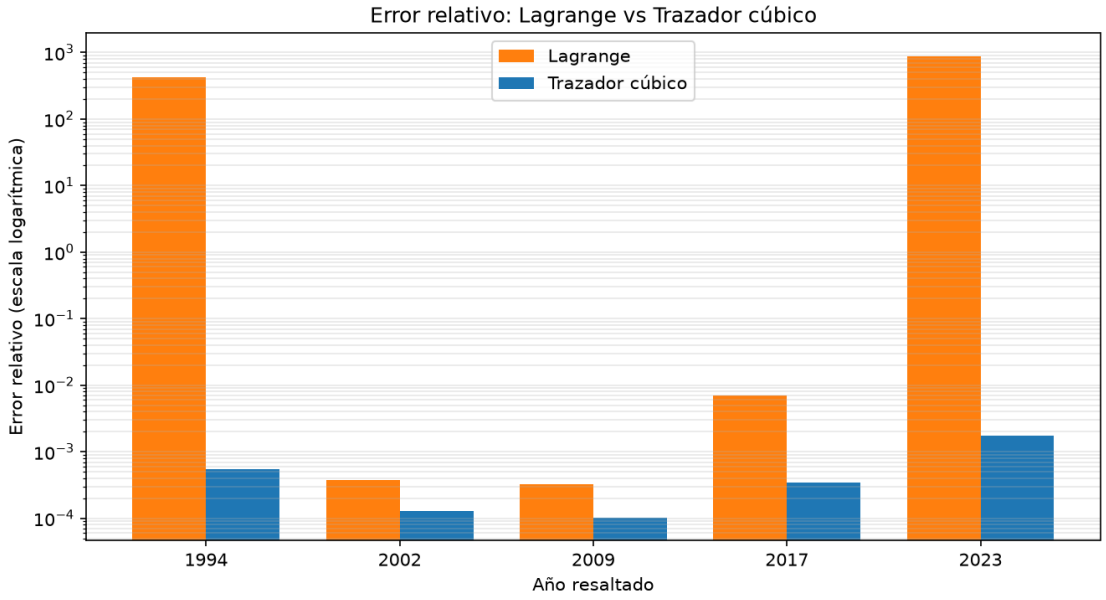


Figura 10: Error relativo de ambos métodos en escala logarítmica.

4. Análisis de resultados

4.1. Qué se observa

- Extrapolación en 1994 y 2023. Estos dos años quedan fuera del rango de los nodos (1995–2022). Lagrange falla por completo: predice poblaciones negativas del orden de -2×10^{10} y -3×10^{10}, con errores relativos de 418 y 886, es decir, de $4{,}2\times10^{4}\,\%$ y $8{,}9\times10^{4}\,\%$. El resultado carece de sentido físico, porque la población real es de unos 52 y 38 millones.

- Interpolación en 2002, 2009 y 2017. Dentro del rango Lagrange aproxima bien, con errores entre 3×10^{-4} y 7×10^{-3} (de 0,03 % a 0,70 %). Aun así, el trazador es mejor en los tres años. En 2017 el error de Lagrange (0,70 %) es unas 20 veces mayor que el del trazador (0,034 %).

- Trazador cúbico. Su error máximo es 0,175 %, en 2023, y en los demás años está por debajo de 0,06 %. Incluso en los años donde extrapola se mantiene cerca de la tendencia de los datos.

La gráfica comparativa muestra que ambas curvas coinciden con los datos en la parte central (aproximadamente 1998–2016). Cerca de los bordes del intervalo, Lagrange empieza a oscilar: se ven ondas antes de 1998 y después de 2018, y entre 1995 y 1996 y entre 2021 y 2022 la curva queda fuera del rango mostrado. El trazador, en cambio, sigue de forma suave la caída de la población de los últimos años.

4.2. Por qué se produjeron estas diferencias

- Naturaleza global frente a local. El polinomio de Lagrange es único y global: cada nodo influye en toda la curva y cualquier pequeña irregularidad en los datos se propaga a todo el intervalo. El trazador es local: cada tramo depende sobre todo de los nodos vecinos, por lo que una irregularidad solo afecta a su entorno.

- Fenómeno de Runge. Carl Runge descubrió en 1901 que cuando los nodos están distribuidos de forma uniforme (equiespaciada), aumentar el grado del polinomio provoca que este oscile con amplitudes crecientes cerca de los extremos del intervalo [2].

  Este comportamiento se explica matemáticamente mediante el término del error de interpolación [1]:

$$f(x) - P(x) = \frac{f^{(n)}(\xi)}{n!} \prod_{k=0}^{n-1} (x - x_k), \qquad \text{con } n = 25.$$

  El producto nodal $\prod_{k=0}^{n-1}(x - x_k)$ adopta valores pequeños en el centro del intervalo, pero sus magnitudes se disparan de forma exponencial conforme x se acerca a los extremos x_0 y x_n. Dado que los registros de población de Ucrania son anuales y están prácticamente equiespaciados ($h \approx 1$ año), un grado tan alto ($n - 1 = 24$) desata exactamente este fenómeno:

  - En la zona central (1998–2016) el polinomio es estable y entrega errores inferiores al 0,04 % (años 2002 y 2009).
  - Cerca de los bordes del rango de datos (1995–1997 y 2020–2022), el polinomio genera crestas y valles artificiales muy pronunciados (visibles en la Figura 5), aumentando el error en el año 2017 a 0,70 %.

  Por fuera de los nodos (1994 y 2023) el polinomio ya no solo oscila, sino que se dispara asintóticamente como x^{24}, explicando los valores absurdos del orden de -10^{10}.

- Cambio de tendencia. La población baja casi de forma lineal hasta 2020 y a partir de 2021 cae con rapidez. Ese cambio de pendiente obliga a un polinomio de grado 24 a oscilar para pasar por todos los nodos, mientras que los tramos cúbicos se adaptan a la curvatura local sin afectar al resto de la curva.

- Grado y condiciones de frontera. En 1994 y 2023 se evalúa el primer y el último tramo, que son cúbicos. La razón principal por la que el valor no diverge es el bajo grado: un cúbico crece como x^3 y no como x^{24}. La condición natural $S'' = 0$ solo fija la curvatura en los extremos y no garantiza una buena extrapolación. Además, solo hay dos años extrapolados, ambos a un año de distancia de los nodos, y el mayor error del trazador (0,175 %) ocurre justo en 2023, así que la extrapolación se vuelve poco fiable si el horizonte es largo.

- Condicionamiento numérico. El cálculo de Lagrange suma términos de magnitud enorme con signos alternos, lo que provoca cancelación y pérdida de cifras significativas. El sistema del trazador es tridiagonal y diagonalmente dominante, por lo que está bien condicionado y su solución es estable.

- Costo. El algoritmo de Thomas resuelve el sistema tridiagonal en $O(n)$ y evaluar el trazador cuesta $O(n)$ por la búsqueda del tramo (o $O(\log n)$ con búsqueda binaria), mientras que evaluar Lagrange cuesta $O(n^2)$ por punto. Con 25 datos la diferencia no se nota, pero crece con n.

5. Conclusiones

1. El trazador cúbico natural fue más preciso que el polinomio de Lagrange en los cinco años de prueba. La diferencia es de varios órdenes de magnitud en 1994 y 2023 y de un factor de 3 a 20 en los años interiores.

2. Interpolar 25 puntos casi equiespaciados con un único polinomio de grado 24 es numéricamente inestable cerca de los bordes del intervalo. Lagrange solo es confiable en el interior del rango de nodos y no debe usarse para extrapolar.

3. En los dos años extrapolados, el trazador también fue mejor (errores de 0,06 % en 1994 y 0,18 % en 2023). Como son solo dos casos, a un año de los nodos, cualquier extrapolación debe tomarse con cautela, porque la curva no tiene información fuera de los nodos.

4. Para series largas, como esta serie demográfica, conviene usar polinomios de grado bajo por tramos, como los trazadores, en lugar de un único polinomio de grado alto.

5. Subir el grado del polinomio no garantiza una mejor aproximación. En teoría (fenómeno de Runge), con nodos equiespaciados el error de Lagrange puede crecer al aumentar el número de nodos, mientras que el del trazador disminuye al refinar la malla. Este proyecto no lo comprobó experimentalmente.

Referencias

[1] R. L. Burden, J. D. Faires, y A. M. Burden, *Numerical Analysis*, 10.ª ed. Boston, MA: Cengage Learning, 2016.

[2] C. Runge, "Über empirische Funktionen und die Interpolation zwischen äquidistanten Ordinaten," *Zeitschrift für Mathematik und Physik*, vol. 46, pp. 224–243, 1901.

[3] L. H. Thomas, "Elliptic Problems in Linear Difference Equations over a Network," Watson Scientific Computing Laboratory, Columbia University, New York, Tech. Rep., 1949.